SOUTH LAKE HOME EDUCATORS



Lesson 12

Hearing It Happen

Semester 1, Week 12 - Sound Effects

At a Glance

Lesson 12 objectives & materials

By the end of today, students will:

- Wire four sound cues into their exact matching code paths
- Recognize why event-driven triggers need no replay guard
- Formalize five lessons' worth of optional sound ideas
- Play a full round and hear every outcome distinctly

What We're Using Today

- Demo laptop with all 8 engines, audio output working
- Godot, for the live lecture demo
- Two new sound assets: sfx_wave.wav, sfx_lose.wav
- Everything else reuses Lessons 2-6's optional sound ideas
- Your own [YourName]Game project from Lesson 11 - not a new one

IGDA framework tie-in: Audio - sound design fundamentals (introductory); Programming - event-driven triggers, code reuse (intermediate).

Today's Agenda



0:00 - 0:05	Welcome back, Lesson 11 recap, the polish phase begins
0:05 - 0:15	The Sound Payoff: Four Sounds, Four Existing Moments
0:15 - 0:35	One Shared Sound for Three Outcomes
0:35 - 0:45	Why None of Today's Sounds Need a Retrigger Guard
0:45 - 0:55	Introduce lab exercise, hand out engine-specific handouts
LAB TIME	***— Hearing It Happen —***
0:55 - 1:50	Wire up all four sounds, test a full round by ear
1:50 - 2:00	Exit check: play a round for the class



Welcome Back

Level 2 Complete is done. The polish phase begins.

5 min

Welcome Back

5 min

Quick recap of Lesson 11:

- Level 2 Complete reached - a real, replayable game with multiple waves
- No new mechanics from here through Lesson 15 - just making the loop feel good
- Today: sound. Next: animation, difficulty scaling, and a final polish pass



Open your own project from last week today, not a new one. Two new sound files are provided - sfx_wave.wav and sfx_lose.wav - everything else may already be in your project from earlier optional ideas.



The Sound Payoff

Four sounds, four moments you already built.

10 min

Four Sounds, Four Existing Moments

10 min



Since Lesson 2, five lessons quietly offered an optional 'add a sound' idea. Today, four sounds become required for everyone - shoot, explosion, wave clear, and game over - matching the capstone's Level 3 feature list exactly.

No new logic today - just one more line at each of four places you already built.

- Shoot - when you fire
- Explosion - when a hit lands
- Wave Clear - a non-final wave ends
- Game Over - lives reach zero



One Shared Sound

Three outcomes, one player, three interchangeable clips.

20 min

Three Outcomes, One Player

20 min

Shoot and hit sounds are easy - one sound each, already wired since Lessons 3-4. The interesting part: win, wave-clear, and lose can never happen at the same time, so they can share ONE sound player.

Godot demo: one AudioStreamPlayer2D, RoundSound, added to the main scene. In the exact branches Lesson 11 already built - true-win, wave-advance, and Lesson 8's lose case - set RoundSound's stream and call play().

One node, three interchangeable clips - the same reuse idea from every lesson this semester, just applied to a sound resource instead of a function. Unity: the same idea with PlayOneShot() and one shared AudioSource.



No Retrigger Guard Needed

Why today's four sounds are safe by construction.

10 min

Why No Retrigger Guard?

10 min



Lesson 2's beep-beep needed an explicit stop call because it was continuous. Today's four sounds are one-shot - they finish on their own. But there's a second reason they're safe: every one lives inside a branch that only runs once per actual event.



TIC-80 note: Lesson 6's optional win sound may have needed a hasWon guard flag back then, since the check ran every frame. Lesson 10/11's restructuring moved it into a one-time hit branch - that old guard flag can be removed now if it's still there.

Good structure, built for one reason, quietly solves a second problem later.



Today's Exercise & Lab

Hearing It Happen - wiring up four sounds.

10 min

Hearing It Happen

10 min

Wire four sounds into the exact branches that already represent their events - built on top of your existing project.

Level 1 - Minimal

All four sounds play at the correct moment, at least once each.

MVP - a complete finish for today.

Level 2 - Complete

All four fire reliably every time, none interfere with each other.

Everything works end to end.

Level 3 - Refined

Add the optional win sound, pitch/volume variety, or tidy the code.

Ready to share.



Play a full round OUT LOUD today - a shot, a hit, an early wave clear, and separately a loss - and confirm all four sound genuinely different from each other.

Lab Time Notes



- Remind students who skipped the optional Lessons 2/3/4/6 ideas that today's Handout gives them full instructions, not just "add two sounds."
- Watch for a sound call placed outside the correct if/else branch - causes two round-ending sounds to play together on the same frame.
- Listen as you circulate this week, not just watch the screen - a wrong or missing sound is often faster to catch by ear than by reading code.
- This is a lighter week - if a cluster finishes early, point them at Level 3 ideas rather than moving ahead to next week's material.



It Sounds Like a Game!

A shot, an explosion, a wave clear, a game over - four moments, four distinct cues.

Next week: basic animation - an explosion effect, a hit flash/reaction.

Keep playtesting a full round at home if you'd like - try a Level 3 sound idea if you haven't already.